

Result Analysis System

Node JS / React JS / Django (CS454PC)

Submitted

In partial fulfilment of the requirements for completion of

Bachelor of Technology

In

Computer Science and Engineering

By

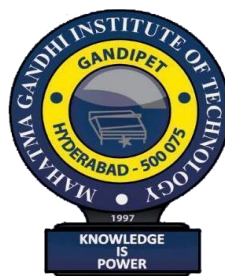
**Srinivas Konala
24261A05R0**

And

**Timothy Saxena
24261A05R4**

Under the guidance of

Ms. Anitha Talari
Assistant Professor
Department of Computer Science and Engineering

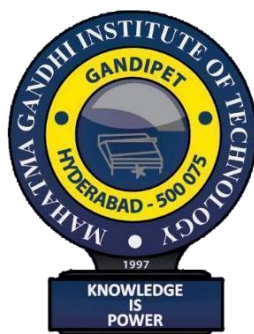


Department of Computer Science and Engineering,
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(A)
GANDIPET, HYDERABAD-500 075, INDIA.

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY(A)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)
Gandipet, Hyderabad-500 075, Telangana (India)



CERTIFICATE

This is to certify that the NODE JS/ REACT JS/ DJANGO (CS454PC) entitled **Result Analysis System**”, is being submitted by **Mr. Srinivas Konala** bearing **Roll No: 24261A05R0** and **Mr. Timothy Saxena** bearing **Roll No: 24261A05R4** in partial fulfilment for completion of **Bachelor of Technology IV Semester in Computer Science and Engineering** to **Mahatma Gandhi Institute of Technology** is a record of Bonafide work carried out by them under our guidance and supervision. The results embodied in this project have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide
Ms. Anitha Talari
Asst. Professor, Dept. of CSE

Head of the Department
Dr. C.R.K Reddy
Professor, Dept. of CSE

DECLARATION

This is to certify that the work reported in NODE JS/ REACT JS/ DJANGO (CS454PC) titled **“Result Analysis System”** is a record of work done by us in the Department of Computer Science and Engineering, Mahatma Gandhi Institute of Technology, Hyderabad. No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

SRINIVAS KONALA
(24261A05R0)

TIMOTHY SAXENA
(24261A05R4)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

We would like to express our sincere thanks to, **Prof G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Dr. C R K Reddy, Professor and HoD, Department of CSE, MGIT**, for all the timely support and valuable suggestions during the period of project.

We are extremely thankful to **Ms. Anitha Talari, Assistant Professor, Department of CSE, MGIT**, NODE JS/ REACT JS/ DJANGO (CS454PC) faculty for her encouragement and support throughout the project.

We are also extremely thankful and indebted to our mentor, **Dr. B. Madhava Rao, Assistant Professor, Department of CSE, MGIT** for his constant guidance, encouragement, and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the CSE Department who helped us directly or indirectly in completing this project.

SRINIVAS KONALA
(24261A05R0)

TIMOTHY SAXENA
(24261A05R4)

TABLE OF CONTENTS

CERTIFICATE	1
DECLARATION	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS	4
LIST OF FIGURES	6
LIST OF TABLES	7
ABSTRACT	8
1. INTRODUCTION	10
2. PROBLEM STATEMENT	11
2.1 Challenges Faced by Students	11
2.2 Challenges Faced by Administration	11
2.3 The Gap This System Fills	11
3. TECHNOLOGIES USED	12
4. SOLUTION	15
4.1 System Architecture	15
4.2 Database Design	15
4.3 Excel-to-MySQL Data Pipeline	16
4.4 Excel-to-MySQL Data Pipeline	17
4.5 Role-Based Access Control	18
4.6 GPA Calculation	19
4.7 Linear Regression SGPA Projection	20
4.8 Student Dashboard — Five-Tab Design	20
4.9 Admin Dashboard Design	21
4.10 Git Workflow and Deployment	21

5. CODE	22
5.1 Import Script — Core Loop	22
5.2 JWT Authentication — Student Login	22
5.3 GPA Calculation Utility	23
5.4 Linear Regression Projection	23
5.5 Auth Middleware	24
5.6 Batch Rank Endpoint	24
6. SCREENSHOTS	25
6.1 Login Page	25
6.2 Student Dashboard Tab	25
6.3 Student Results Tab	26
6.4 Student GPA Overview Tab — Actual Trend	26
6.5 Student GPA Overview Tab — Projection	27
6.6 Student Failed Subjects Tab	28
6.7 Student Marksheet Tab and PDF Output	28
6.8 Admin Dashboard Overview	29
6.9 Admin Toppers / Ranklist	30
6.10 Admin Subject Analysis	31
6.11 Admin CGPA Distribution	31
7. FUTURE ENHANCEMENTS	32
7.1 Student Name Integration	32
7.2 Password Change Feature	32
7.3 Projection Model Improvement	32
7.4 Email Notifications	32
7.5 Mobile Application	32
7.6 Attendance Integration	32
8. CONCLUSION	33

LIST OF FIGURES

FIGURE	PAGE NO
Figure 4.1: Three-Tier Architecture Diagram	22
Figure 4.2: Entity-Relationship Diagram	23
Figure 4.3: Import Pipeline Flow	25
Figure 4.4: JWT Authentication Flow	26
Figure 4.5: Linear Regression SGPA Projection Chart	27
Figure 6.1: Login Page — Role tab switcher with Student and Admin tabs	32
Figure 6.2: Student Dashboard Tab — Stat cards, batch rank card,	33
Figure 6.3: Student Results Tab — Semester-grouped table	34
Figure 6.4: GPA Overview Tab — Cumulative CGPA trend	35
Figure 6.5: GPA Overview Tab — Linear regression projection	36
Figure 6.6: Failed Subjects Tab — Backlog list with per-semester filter	37
Figure 6.7: Marksheet Tab — Per-semester download cards	37
Figure 6.7.1: Generated PDF Marksheet	37
Figure 6.8: Admin Dashboard — Department stat cards, section breakdown	37
Figure 6.9: Admin Toppers — CGPA-ranked student list with section	37
Figure 6.10: Admin Subject Analysis — Per-subject statistics table	37
Figure 6.11: Admin CGPA Distribution — Histogram with 0.5-step buckets	37

LIST OF TABLES

TABLE	PAGE NO
Table 3.1: Technologies and their roles in the project	17
Table 4.1: Database schema overview	23
Table 4.2: Backend API route reference	23

ABSTRACT

The Result Analysis System is a full-stack web application developed for the Department of Computer Science and Engineering at Mahatma Gandhi Institute of Technology. The system addresses the absence of a centralised platform through which students can view and analyse their academic results, and administrators can monitor performance across the department.

The application supports two user roles. Students are provided with a five-tab personal dashboard covering a summary overview, subject-wise results with search and filter, a GPA overview with a cumulative CGPA trend chart and a linear regression-based SGPA projection for Year 3 and Year 4 students, a dedicated failed subjects tab, and a marksheet download facility. The Admin role provides a department-wide analytics platform covering section-wise performance, semester-wise trends, topper rankings with CGPA and SGPA-based filtering, subject-level analysis, and a CGPA distribution histogram.

The system was built using Node.js and Express.js for the backend, MySQL as the relational database, and React with Vite for the frontend. Authentication is implemented with JSON Web Tokens and bcrypt password hashing. Role-based access control is enforced at the middleware layer. The dataset comprises results for approximately 1,090 students across 35,582 result rows spanning multiple semesters, imported from departmental Excel files using a custom Node.js pipeline.

The application is fully deployed in the cloud: the React frontend on Vercel, the Express backend on Render, and the MySQL database on Railway. It is accessible through a live public URL at <https://result-analysis-three.vercel.app> and requires no local installation.

Keywords: *Result Analysis, Academic Performance, Node.js, Express.js, MySQL, React, JWT, Role-Based Access Control, Linear Regression, GPA Projection, Cloud Deployment*

1. INTRODUCTION

The Result Analysis System is a multi-role academic performance management platform designed for the CSE Department at MGIT. It replaces the traditional approach of distributing printed marksheets and maintaining results in disconnected Excel spreadsheets with a centralised web application that provides instant, authenticated access to results, automated GPA calculations, performance projections, and department-wide analytics.

The application was developed as a team project by two IV Semester students as part of the CS454PC course on Node JS / React JS / Django. The project covers a realistic academic dataset of over 35,000 result entries across five sections (CSE-1 through CSE-5) and multiple semesters, representative of actual department-scale data management requirements.

The system is structured around two user roles. The Student role provides a rich personal academic dashboard with five dedicated tabs: a summary overview with batch rank, a results browser, a GPA overview with trend analysis and future projections, a failed subjects tracker, and a marksheet generator. The Admin role provides a comprehensive department-wide analytics platform enabling data-driven oversight of academic performance.

Beyond functional features, the project demonstrates core engineering concepts from the CS454PC curriculum: RESTful API design with Express.js, relational database modelling with MySQL, client-side rendering with React, stateless JWT authentication, statistical projection using linear regression, and full-stack deployment across cloud platforms.

2. PROBLEM STATEMENT

Academic result management in most engineering departments continues to rely on printed marksheets, individually maintained Excel files, and periodic notice board displays. This approach has significant shortcomings that affect both students and the department administration.

2.1 Challenges Faced by Students

Students have no on-demand access to their complete academic record. Marksheets are distributed physically and are easily misplaced. There is no automated calculation of SGPA or CGPA, and students who need to estimate their performance in future semesters have no data-driven tool to do so. Failed subjects are not tracked centrally, and there is no single view of academic performance trends across semesters.

2.2 Challenges Faced by Administration

The department has no consolidated view of academic performance. Comparing pass percentages across sections, identifying subjects with consistently high failure rates, generating a department-wide topper list, or producing a CGPA distribution of the batch requires manually aggregating data from multiple sources. This makes academic monitoring slow and reactive rather than proactive.

2.3 The Gap This System Fills

The Result Analysis System addresses all of the above by providing a single authenticated web application that centralises result data, automates GPA computation, enables future semester performance projection using linear regression, and makes real-time analytics accessible to the admin without any spreadsheet work. The system converts a fragmented manual process into a coherent, data-driven workflow accessible from any device through a web browser.

3. TECHNOLOGIES USED

3.1 Node.js

Node.js is a JavaScript runtime environment built on Google Chrome's V8 engine. It executes JavaScript on the server side using a non-blocking, event-driven I/O model suited for handling concurrent API requests efficiently. In this project, Node.js serves as the runtime for the entire backend application.

3.2 Express.js

Express.js is a minimal web application framework for Node.js. It provides an API for defining HTTP routes, applying middleware, and handling requests and responses. The backend is structured as an Express application with route files for authentication, student operations, and admin analytics. Middleware handles CORS, JSON body parsing, and JWT-based route protection.

3.3 MySQL

MySQL is an open-source relational database management system. The project uses four tables: students, results, faculty, and admins. The results table contains over 35,000 rows. MySQL's support for JOIN queries, aggregate functions, and GROUP BY operations is central to GPA calculations and analytics. The database is hosted on Railway and connected via a connection pool using the mysql2 library.

3.4 React and Vite

React is a JavaScript library for building user interfaces through reusable components and a virtual DOM. Vite is the build tool and development server used to bundle the React application. The frontend uses a custom client-side router implemented directly in App.jsx using window.location.pathname, keeping the dependency footprint minimal. All five student dashboard tabs and both admin dashboard pages are React components.

3.5 JSON Web Tokens (JWT)

JSON Web Tokens are used for stateless authentication. On successful login, the server signs a token containing the user's ID and role. This token is stored in the browser's localStorage and attached to every API request as a Bearer token in the Authorization header. The server verifies the signature on each request without a database lookup, making authentication scalable and sessionless.

Technology	Role	Where Used
Node.js	Server-side runtime	Backend
Express.js	HTTP framework & routing	Backend
MySQL	Relational database	Railway (cloud)
React + Vite	Frontend UI library & build tool	Frontend (Vercel)
JWT (jsonwebtoken)	Stateless authentication tokens	Backend + Frontend
bcryptjs	Password hashing	Backend
pdfkit	PDF marksheet generation	Backend
json2csv	CSV data export	Backend
xlsx	Excel file parsing	Import script
Vercel	Frontend static hosting + CDN	Cloud
Render	Backend Node.js hosting	Cloud
Railway	Managed MySQL hosting	Cloud

Table 3.1: Technologies and their roles in the project

3.6 bcryptjs

bcryptjs is used for one-way password hashing. Passwords are never stored in plain text. During import, each student's default password is hashed with bcrypt at a salt round of 10. At login, bcrypt's compare function tests the submitted password against the stored hash, returning true or false without ever reversing the hash.

3.7 pdfkit

pdfkit is a Node.js library for generating PDF documents programmatically without requiring a browser or headless Chrome. It is used to generate semester marksheets on demand. The backend constructs a complete A4 PDF in memory — including the college header, subject table, SGPA summary boxes, and a pass/fail status banner — then streams it to the browser as a downloadable file.

3.8 json2csv

json2csv converts JavaScript arrays of objects into CSV-formatted strings. It is used by the admin export routes to allow bulk download of filtered result data in spreadsheet-compatible format.

3.9 xlsx

The xlsx library reads Excel files in Node.js. It is used exclusively in the import script to parse departmental result files, converting each worksheet row into a JavaScript object for database insertion.

3.10 Deployment Platforms

Three cloud platforms handle deployment. Vercel hosts the React frontend, building and deploying it automatically on every push to the dev branch. Render hosts the Node.js backend as a persistent web service with environment variables for database credentials and the JWT secret. Railway hosts the MySQL database and provides a publicly accessible connection endpoint. All three services are connected to the same GitHub repository and deploy automatically when the dev branch is updated.

4. SOLUTION

4.1 System Architecture

The system follows a three-tier architecture. The presentation tier is a React application hosted on Vercel that communicates with the backend through HTTP API calls only. The application tier is a Node.js Express server hosted on Render that handles all business logic including authentication, GPA calculation, linear regression, and PDF generation. The data tier is a MySQL database hosted on Railway. No tier has direct knowledge of another tier's internal implementation, allowing each to be modified independently.

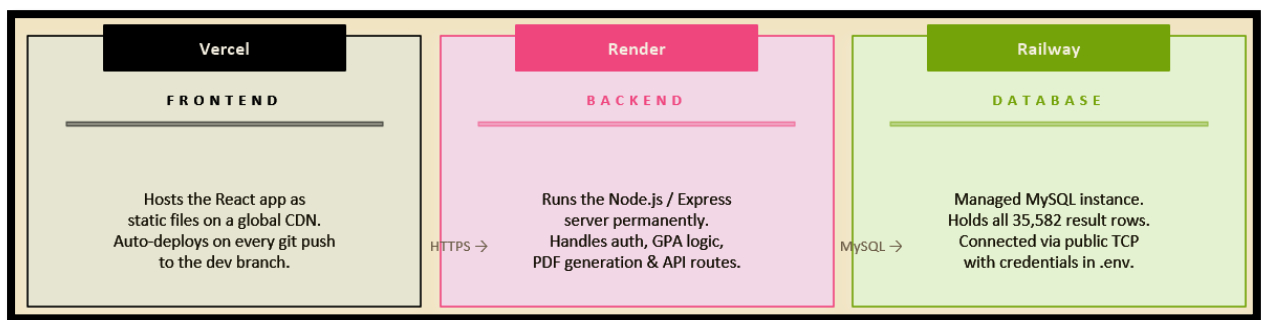


Figure 4.1: Three-Tier Architecture Diagram: Browser → Vercel (React) → Render (Express) → Railway (MySQL)

4.2 Database Design

The database consists of three tables. The students table stores one row per student with their hall ticket number as the primary key, a bcrypt password hash, and their assigned branch, year, and section. The results table is the core data store — each row represents a single student's performance in one subject in one semester. A unique constraint on (ht_no, semester, course_code) prevents duplicate entries and enables idempotent re-import. The faculty table stores accounts each scoped to a single course code. The admins table stores admin credentials with unrestricted access.

Table	Primary Key	Key Columns	Purpose
students	ht_no	password_hash, section, year, branch	1 row per student
results	id (auto)	ht_no (FK), semester, course_code, total_marks, grade_letter, credits	1 row per student-subject-semester
admins	id (auto)	username, password_hash	Admin accounts with full access

Table 4.1: Database schema overview

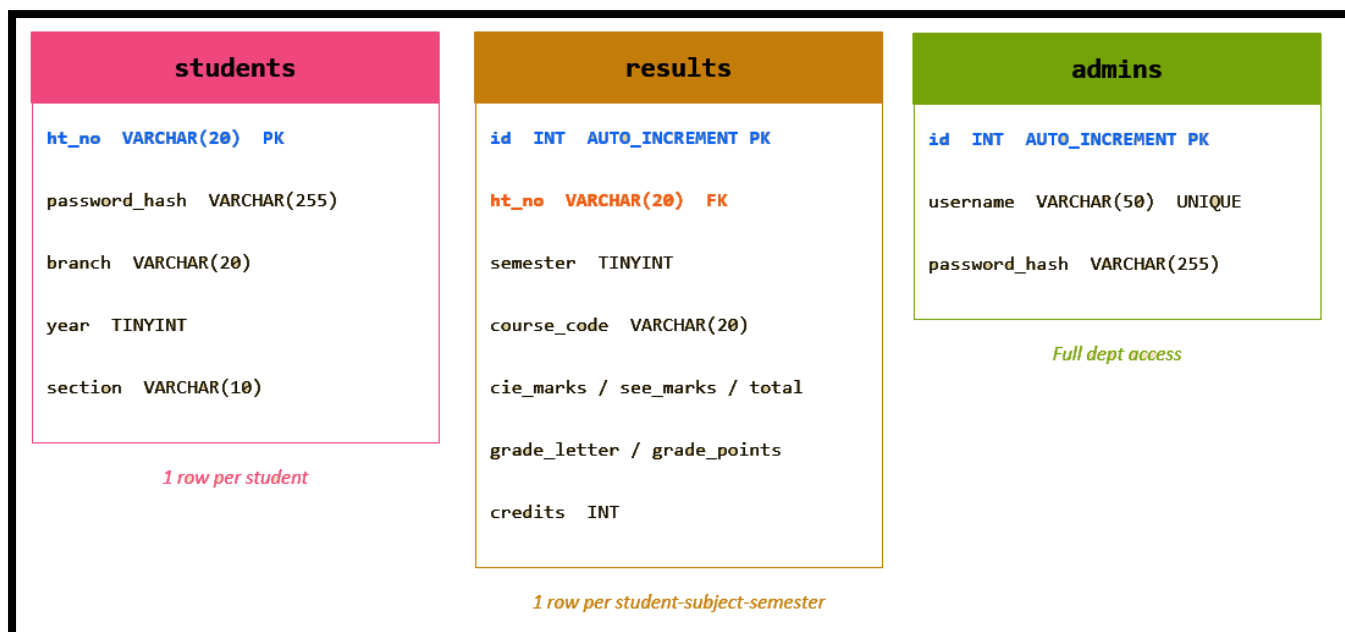


Figure 4.2: Entity-Relationship Diagram: students $\longleftrightarrow$ results, faculty, and admins tables with key columns and constraints

4.3 Excel-to-MySQL Data Pipeline

The department supplied result data as Excel files — one file per semester with approximately 3,000 rows each in the format: HT_NO, COURSE_CODE, COURSE_NAME, CIE_MARKS, SEE_MARKS, TOTAL_MARKS, GRADE_LETTER, GRADE_POINTS, CREDITS. A custom Node.js script (backend/scripts/import.js) was written to process these files and populate the database.

The script accepts two command-line arguments: the Excel file path and the semester number. The semester number is passed manually because it is not present in the Excel file itself. For each row, the script checks whether the student already exists; if not, it creates a new student record with a default password hashed with bcrypt. The result row is inserted using ON DUPLICATE KEY UPDATE, making the script safely idempotent. The semester number is passed manually because it is not present in the Excel file itself. For each row, the script checks whether the student already exists; if not, it creates a new student record with a default password hashed with bcrypt. Section assignment is inferred from the last two characters of the HT number using the department's roll range boundaries.

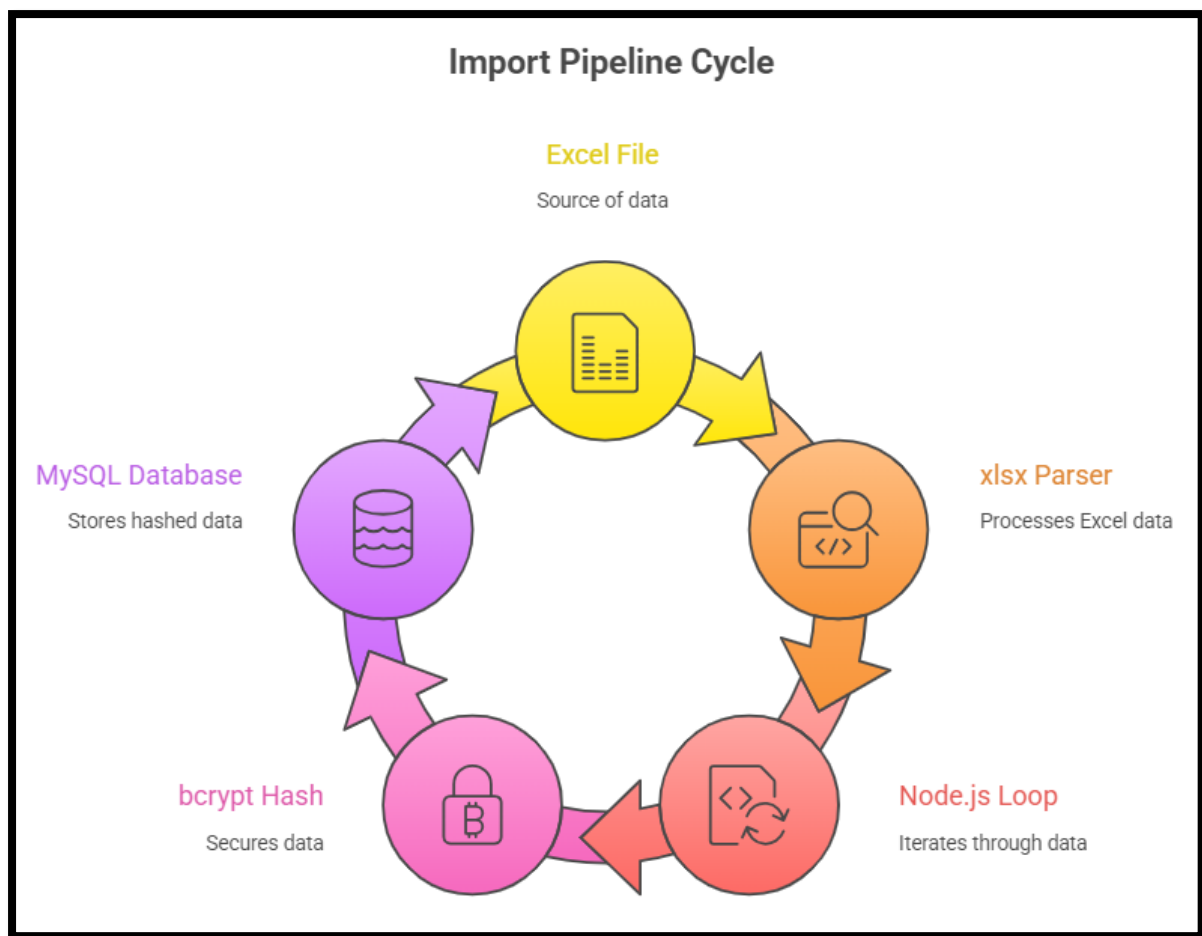


Figure 4.3: Import Pipeline Flow: Excel File → xlsx parser → Node.js loop → bcrypt hash → MySQL INSERT

4.4 API Route Design

The backend exposes RESTful endpoints grouped by role. Every protected route passes through a JWT verification middleware followed by a role guard.

Method	Endpoint	Role	Purpose
POST	/api/auth/student-login	Public	Student login, returns JWT
POST	/api/auth/admin-login	Public	Admin login, returns JWT
GET	/api/student/results	Student	Fetch all results, optional ?semester filter
GET	/api/student/sgpa	Student	SGPA per semester for trend chart
GET	/api/student/cgpa	Student	Overall CGPA and failed count
GET	/api/student/failed	Student	All failed subjects
GET	/api/student/rank	Student	Batch rank and percentile
GET	/api/student/marksheet/:semester	Student	Download PDF marksheet

GET	/api/admin/dashboard	Admin	Dept overview, section & semester tables
GET	/api/admin/toppers	Admin	Ranked list by CGPA or SGPA
GET	/api/admin/subject-analysis	Admin	Per-subject stats
GET	/api/admin/cgpa-distribution	Admin	CGPA histogram data
GET	/api/admin/section/:section	Admin	Drill-down for a specific section
GET	/api/admin/export	Admin	Full CSV export with optional filters

Table 4.2: Backend API route reference

4.5 Role-Based Access Control

Each role has a precisely defined data scope enforced at the server level. A student's JWT contains their HT number; every student route queries using `req.user.id`, making it impossible for one student to access another's records even if they manipulate the request. The admin role has no scope restriction and can query across all students, sections, and semesters.

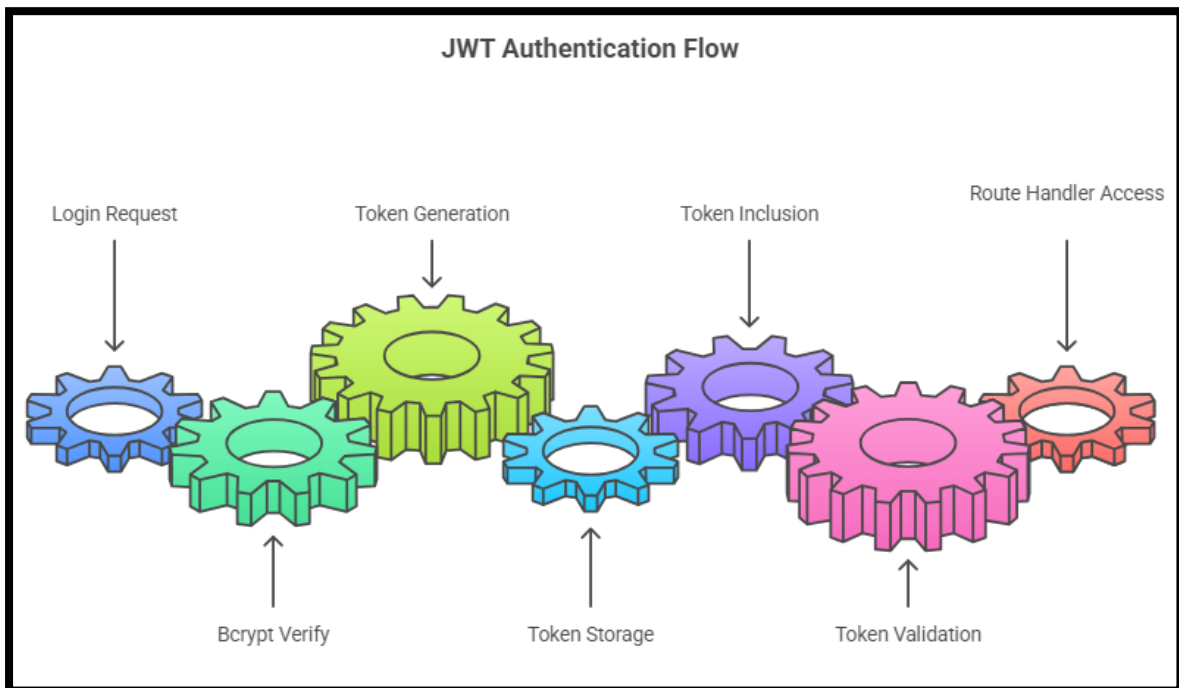


Figure 4.4: JWT Authentication Flow: Login Request → bcrypt verify → jwt.sign → Token stored → Bearer header → jwt.verify → Route handler

4.6 GPA Calculation

GPA calculation is implemented as a shared utility in backend/utills/gpa.js. The SGPA for a given set of results is the sum of (grade_points × credits) divided by total credits, excluding subjects with zero credits. CGPA applies the same formula across all semesters combined. Both student-facing routes and admin analytics use this single shared function, ensuring GPA values are consistent regardless of which part of the application computes them.

4.7 Linear Regression SGPA Projection

The GPA Overview tab includes a forward-looking SGPA projection for Year 3 and Year 4 students who have sufficient historical data. The projection is computed using simple linear regression applied to the student's existing SGPA values across semesters.

In linear regression, given a set of data points (x_i, y_i) where x is the semester number and y is the SGPA, the best-fit line is determined by calculating the slope (m) and intercept (b) as follows:

$$m = (n \times \Sigma(x \times y) - \Sigma x \times \Sigma y) / (n \times \Sigma x^2 - (\Sigma x)^2)$$

$$b = (\Sigma y - m \times \Sigma x) / n$$

Where n is the number of semesters completed. The projected SGPA for future semesters is then computed as: $\text{projected_SGPA} = m \times \text{semester_number} + b$. The projected values are capped between 0 and 10. The chart displays actual SGPA values as a solid line and projected values as a dashed line in a different colour, making the boundary between historical data and prediction.



Figure 4.5: Linear Regression SGPA Projection Chart: Actual SGPA (solid line) vs Projected SGPA (dashed line)

4.8 Student Dashboard — Five-Tab Design

The student interface is organised into five tabs to avoid information overload while keeping all features accessible from a single page.

Tab 1: Dashboard

The Dashboard tab serves as the landing screen after login. It displays the MGIT logo, a welcome message with the student's HT number, and four summary stat cards: overall CGPA, latest SGPA, number of semesters completed, and backlog count. A batch rank card shows the student's rank within their batch computed by the `/api/student/rank` endpoint. The most recent semester's results are displayed in a summary table below the cards.

Tab 2: Results

The Results tab displays all subject results grouped by semester. A semester dropdown filter and a text search box allow the student to quickly find specific subjects or narrow down to a particular term. Each semester group shows the SGPA for that term. Failed subjects are visually highlighted with a red indicator and a faint red row background.

Tab 3: GPA Overview

The GPA Overview tab contains two charts. The first is a cumulative CGPA trend line showing how the student's overall CGPA has evolved as each semester was completed. The second chart shows the SGPA trend with the linear regression projection for Year 3 and Year 4 students. Both charts are rendered as pure SVG with no external charting dependencies. A table below the charts lists SGPA per semester alongside grade statistics.

Tab 4: Failed Subjects

The Failed Subjects tab provides a dedicated view of all subjects in which the student has received an F grade. A semester filter allows the student to check whether failed subjects belong to a specific term. Each entry shows the subject code, subject name, semester, marks obtained, and the grade. This tab is intentionally separated from the main results view to make backlogs immediately visible without scrolling through a full results table.

Tab 5: Marksheet

The Marksheet tab displays one download card per completed semester. Each card shows the semester number and the SGPA for that term. Clicking the download button triggers a request to the backend, which generates an A4 PDF marksheet using `pdfkit` and streams it to the browser. The PDF includes the college header, student details, a subject table, summary boxes, and a colour-coded pass/fail banner.

4.9 Admin Dashboard Design

The admin interface is a single-page application with a persistent left sidebar for navigation. Four tabs are available: Department Overview, Toppers / Ranklist, Subject Analysis, and CGPA Distribution. Sidebar filters for semester, section, and year apply to whichever tab is currently active. A CSV export button triggers a full data download with the active filters applied.

4.10 Git Workflow and Deployment

Development followed a structured Git branching strategy. The main branch holds stable, tested code. The dev branch is the integration branch and is the branch from which both Vercel and Render deploy automatically on push. Feature work is done on short-lived feature branches created off dev, which are merged back via pull requests after review by the second team member.

The deployment pipeline requires no manual steps after a push. Vercel detects the push to dev, runs `npm run build` to produce the static React bundle, and serves it from a global CDN with a custom SPA rewrite rule in `vercel.json`. Render detects the push, runs `npm install`, and restarts the Express server. Railway requires no redeployment as the database persists independently of application deployments.

5. CODE

This chapter presents key code excerpts from the project. The full source code is available at <https://github.com/timothy-saxena/result-analysis> on the dev branch.

5.1 Import Script — Core Loop (backend/scripts/import.js)

The import script processes one Excel file per run. For each row it creates a student account if one does not exist and upserts the result record using ON DUPLICATE KEY UPDATE, making repeated runs safe.

```
for (const row of rows) {
  const ht_no = String(row["HT_NO"]).trim();

  // Create student if not already present
  const [existing] = await db.query(
    "SELECT ht_no FROM students WHERE ht_no = ?", [ht_no]
  );
  if (existing.length === 0) {
    const hash = await bcrypt.hash(`${ht_no}@123`, 10);
    const meta = inferStudentMeta(ht_no);
    await db.query(
      "INSERT INTO students VALUES (?, ?, ?, ?, ?)",
      [ht_no, hash, meta.branch, meta.year, meta.section]
    );
  }

  // Upsert result - safe to re-run on the same file
  await db.query(
    `INSERT INTO results
      (ht_no, semester, course_code, course_name,
       cie_marks, see_marks, total_marks,
       grade_letter, grade_points, credits)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE
        total_marks = VALUES(total_marks),
        grade_letter = VALUES(grade_letter),
        grade_points = VALUES(grade_points)\`,
    [ht_no, semester, courseCode, /* ... */]
  );
}
```

5.2 JWT Authentication — Student Login (backend/routes/auth.js)

The login route queries the student record, compares the submitted password against the stored bcrypt hash, and returns a signed JWT on success.

```
router.post("/student-login", async (req, res) => {
  const { ht_no, password } = req.body;
  const [rows] = await db.query(
    "SELECT * FROM students WHERE ht_no = ?", [ht_no]
  );
  if (rows.length === 0)
    return res.status(401).json({ error: "Invalid credentials." });

  const valid = await bcrypt.compare(password, rows[0].password_hash);
```

```

if (!valid)
  return res.status(401).json({ error: "Invalid credentials." });

const token = jwt.sign(
  { id: ht_no, role: "student" },
  process.env.JWT_SECRET,
  { expiresIn: "7d" }
);
res.json({ token, role: "student", ht_no });
});

```

5.3 GPA Calculation Utility (backend/utils/gpa.js)

The SGPA function is the shared source of truth used by both student routes and admin analytics.

```

const calculateSGPA = (results) => {
  let totalWeighted = 0;
  let totalCredits = 0;
  for (const row of results) {
    if (row.credits > 0) {
      totalWeighted += row.grade_points * row.credits;
      totalCredits += row.credits;
    }
  }
  if (totalCredits === 0) return 0;
  return parseFloat((totalWeighted / totalCredits).toFixed(2));
};

// CGPA = same formula applied across all semesters combined
const calculateCGPA = (allResults) => calculateSGPA(allResults);

```

5.4 Linear Regression Projection (frontend — GPA Overview tab)

The following function computes the slope and intercept of the best-fit line through the student's historical SGPA values, then projects SGPA for future semesters.

```

function linearRegression(points) {
  const n = points.length;
  const sumX = points.reduce((a, p) => a + p.x, 0);
  const sumY = points.reduce((a, p) => a + p.y, 0);
  const sumXY = points.reduce((a, p) => a + p.x * p.y, 0);
  const sumX2 = points.reduce((a, p) => a + p.x * p.x, 0);
  const m = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
  const b = (sumY - m * sumX) / n;
  return { m, b };
}

function projectSGPA(sgpaData, futureSemesters) {
  const points = sgpaData.map(s => ({ x: s.semester, y: s.sgpa }));
  const { m, b } = linearRegression(points);
  return futureSemesters.map(sem => ({
    semester: sem,
    projected: Math.min(10, Math.max(0,
      parseFloat((m * sem + b).toFixed(2))
    )),
  }));
}

```

5.5 Auth Middleware (backend/middleware/auth.js)

Every protected route passes through `verifyToken` and `requireRole` before the handler runs.

```
const verifyToken = (req, res, next) => {
  const token = req.headers["authorization"]?.split(" ")[1];
  if (!token)
    return res.status(401).json({ error: "No token provided." });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(403).json({ error: "Invalid or expired token." });
  }
};

const requireRole = (roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: "Access denied." });
  next();
};
```

5.6 Batch Rank Endpoint (backend/routes/student.js)

The `/student/rank` endpoint computes the student's rank within their batch by computing CGPA for every student and counting how many have a higher value.

```
router.get("/rank", async (req, res) => {
  const ht_no = req.user.id;

  // Get all students in the same batch (same year prefix)
  const batchPrefix = ht_no.substring(0, 2);
  const [allStudents] = await db.query(
    `SELECT r.ht_no,
      ROUND(SUM(r.grade_points * r.credits) /
        NULLIF(SUM(CASE WHEN r.credits > 0
          THEN r.credits ELSE 0 END), 0), 2) AS cgpa
      FROM results r
      JOIN students s ON r.ht_no = s.ht_no
      WHERE r.ht_no LIKE ? AND r.credits > 0
      GROUP BY r.ht_no`,
    [`${batchPrefix}%`]
  );

  const myCGPA = allStudents.find(s => s.ht_no === ht_no)?.cgpa || 0;
  const rank = allStudents.filter(s => s.cgpa > myCGPA).length + 1;
  const total = allStudents.length;

  res.json({ rank, total,
    percentile: parseFloat(((1 - rank / total) * 100).toFixed(1)),
    cgpa: myCGPA
  });
});
```


6. SCREENSHOTS

The following screenshots show the key screens of the Result Analysis System from the live deployment at <https://result-analysis-three.vercel.app>.

6.1 Login Page

The login page presents role tabs for Student and Admin. Each tab adjusts the identifier field label, placeholder, and the backend endpoint called on submission. On successful authentication the browser is redirected to the role-appropriate dashboard.

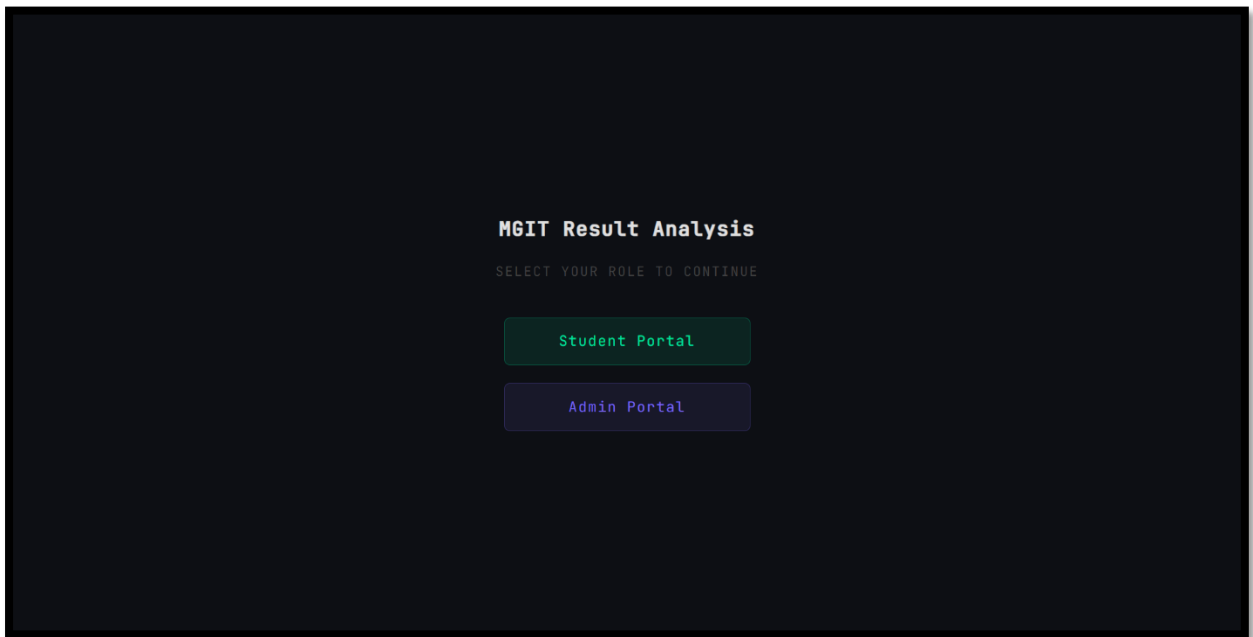


Figure 6.1: Login Page — Role tab switcher with Student and Admin tabs, identifier and password fields

6.2 Student Dashboard Tab

The Dashboard tab shows the MGIT logo, the student's HT number, four summary stat cards (CGPA, latest SGPA, semesters completed, backlog count), a batch rank card showing the student's rank and percentile within their batch, and a summary table of the most recent semester's results.

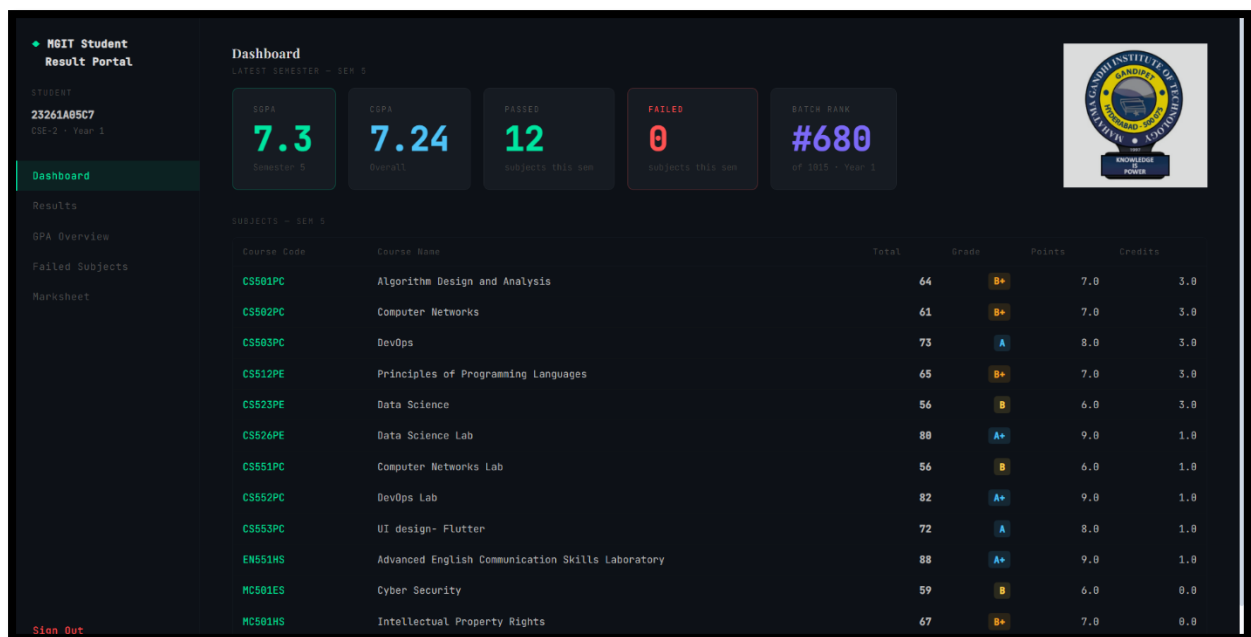


Figure 6.2: Student Dashboard Tab — Stat cards, batch rank card, and latest semester results table

6.3 Student Results Tab

The Results tab displays all subject results grouped by semester. A text search box and semester filter allow targeted browsing. Failed subjects are marked with a red indicator. Each semester group header shows the corresponding SGPA.

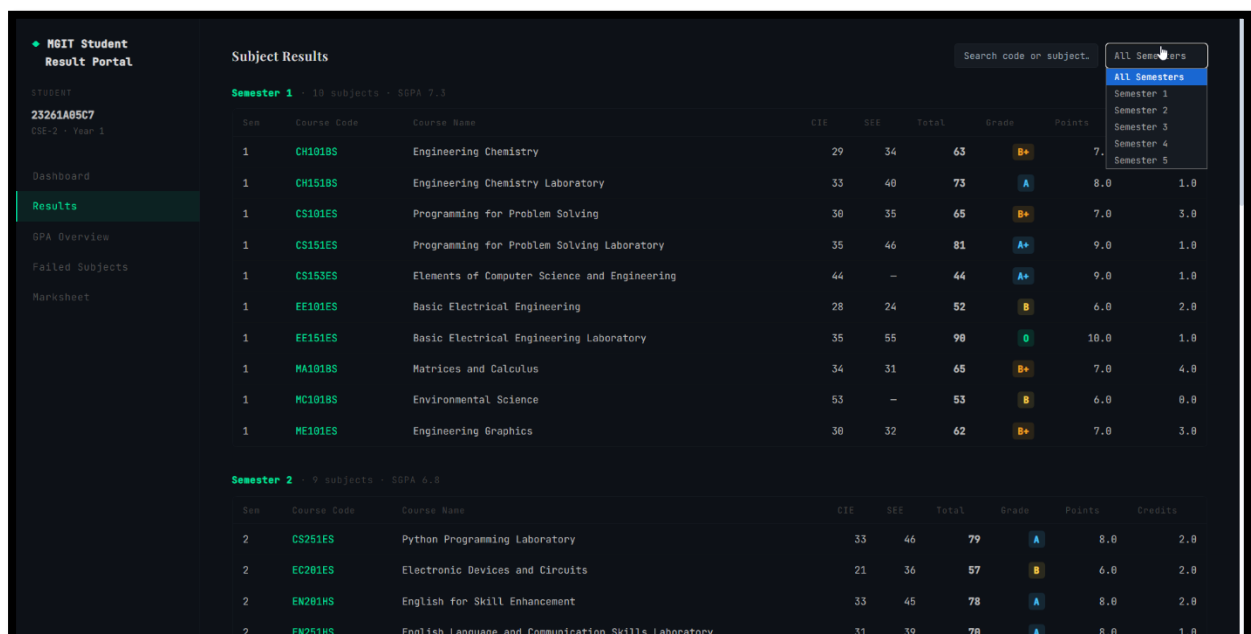


Figure 6.3: Student Results Tab — Semester-grouped table with search filter and grade badges

6.4 Student GPA Overview Tab — Actual Trend

The GPA Overview tab shows a cumulative CGPA trend line and a semester-wise SGPA chart. Both are rendered as pure SVG. The actual SGPA values are plotted as a solid line with labelled

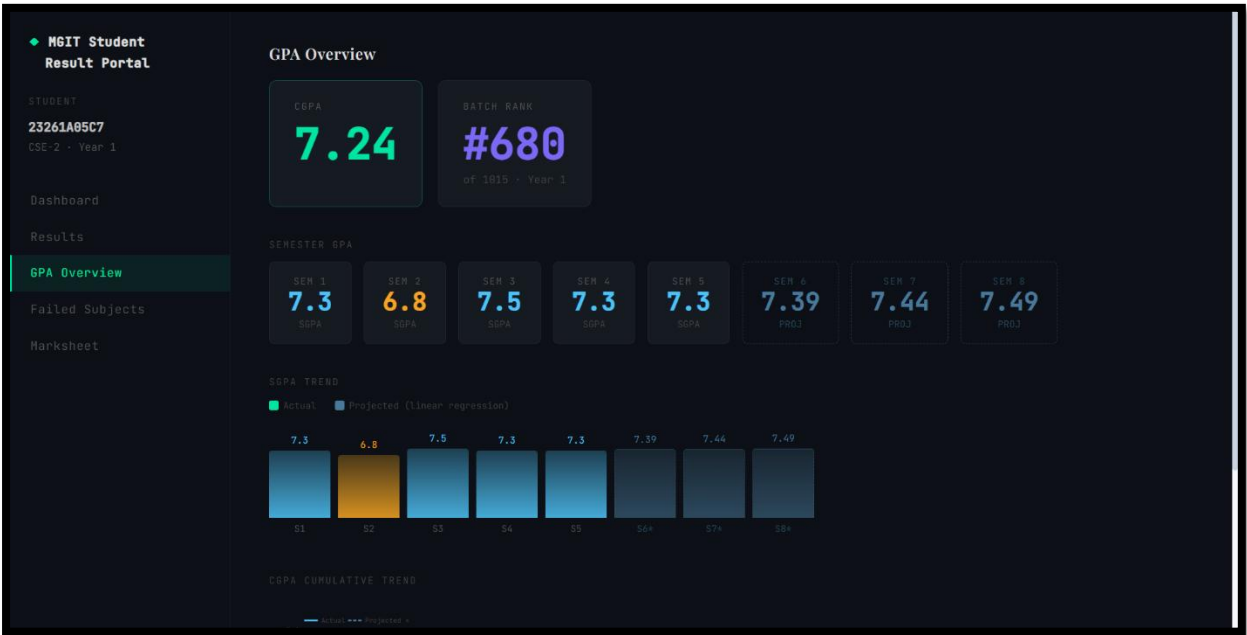


Figure 6.4: GPA Overview Tab — Cumulative CGPA trend and semester-wise SGPA chart (actual data)

6.5 Student GPA Overview Tab — Projection

For Year 3 and Year 4 students with sufficient historical data, the SGPA chart extends into future semesters using a linear regression projection. The projected portion is rendered as a dashed line in a distinct colour, clearly separating historical data from the statistical forecast.



Figure 6.5: GPA Overview Tab — Linear regression projection shown as dashed line beyond actual semesters

6.6 Student Failed Subjects Tab

The Failed Subjects tab lists all subjects in which the student has received an F grade. A semester filter narrows the view to a specific term. Each entry shows subject code, subject name, semester, marks obtained, and grade.

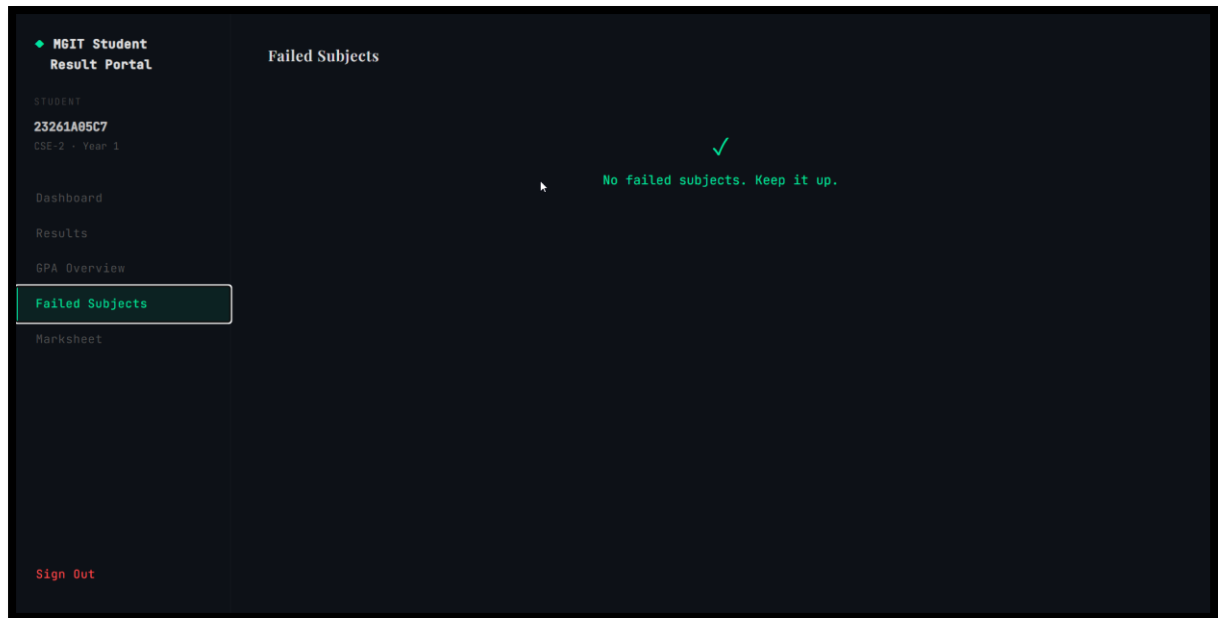


Figure 6.6: Failed Subjects Tab — Backlog list with per-semester filter

6.7 Student Marksheet Tab and PDF Output

The Marksheet tab shows one download card per semester with the SGPA for that term. Clicking download generates a formatted A4 PDF using pdftkit. The PDF includes the college header, student details, a full subject table, an SGPA summary row, and a colour-coded pass/fail banner.

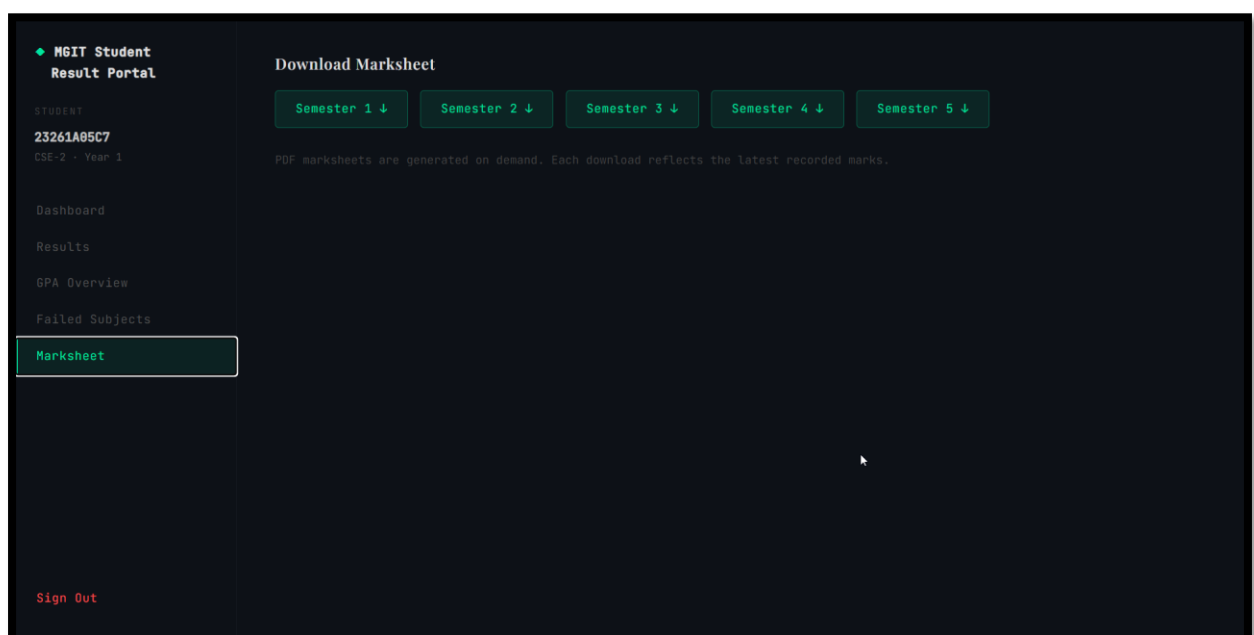


Figure 6.7: Marksheet Tab — Per-semester download cards

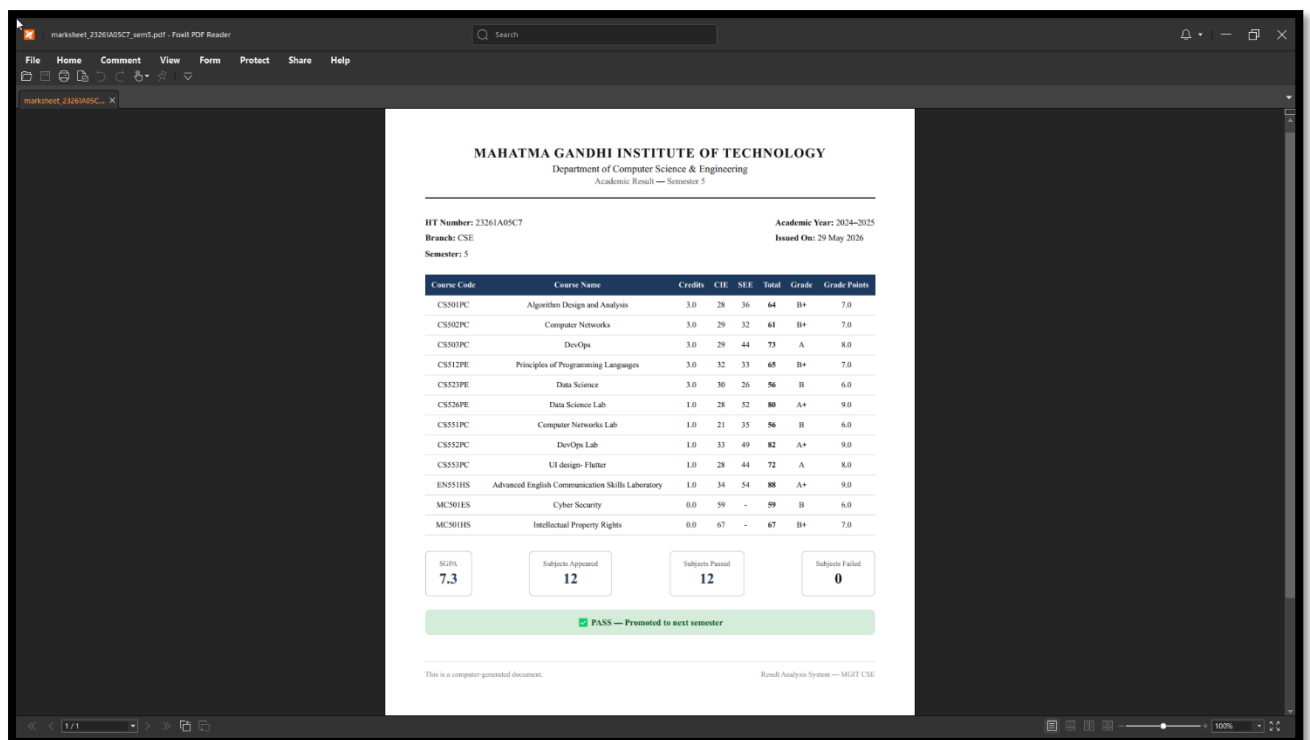


Figure 6.7.1: Generated PDF Marksheet — A4 layout with subject table, SGPA summary, and pass/fail banner

6.8 Admin Dashboard Overview

The Admin Dashboard tab shows four stat cards (total students, total results, total pass, total fail), a section breakdown table comparing CSE-1 through CSE-5, and a semester-wise performance table with average marks and pass percentages.

MGIT Results Portal

ADMIN

Dashboard

Toppers / Ranklist

Subject Analysis

CGPA Distribution

FILTERS

All Semesters

All Sections

All Years

Export CSV

Department Overview

Total Students1090

Total Results35582

Total Pass33557

Total Fail2025

SECTION BREAKDOWN

Section	Students	Avg Marks	Pass	Fail	Pass %
CSE-1	329	71.68	11073	588	0.1%
CSE-2	251	72.23	9882	533	0.1%
CSE-3	256	70.62	9164	536	0.1%
CSE-4	128	69.80	2129	218	0.1%
CSE-5	126	69.73	2109	150	0.1%

SEMESTER-WISE PERFORMANCE

Semester	Students	Avg Marks	Pass	Fail	Pass %
Sem 1	1015	67.54	9064	748	92.38%
Sem 2	691	74.88	5803	398	93.58%
Sem 3	764	72.07	6447	408	94.05%
Sem 4	416	67.50	3916	201	95.12%
Sem 5	411	73.68	4736	177	96.40%
Sem 6	284	74.66	1986	74	96.41%

Figure 6.8: Admin Dashboard — Department stat cards, section breakdown, and semester performance table

6.9 Admin Toppers / Ranklist

The Toppers tab generates a ranked list by CGPA by default. Selecting a semester switches to SGPA-based ranking. Section and year filters allow cohort-specific drill-down. Tied GPA values receive the same rank number.

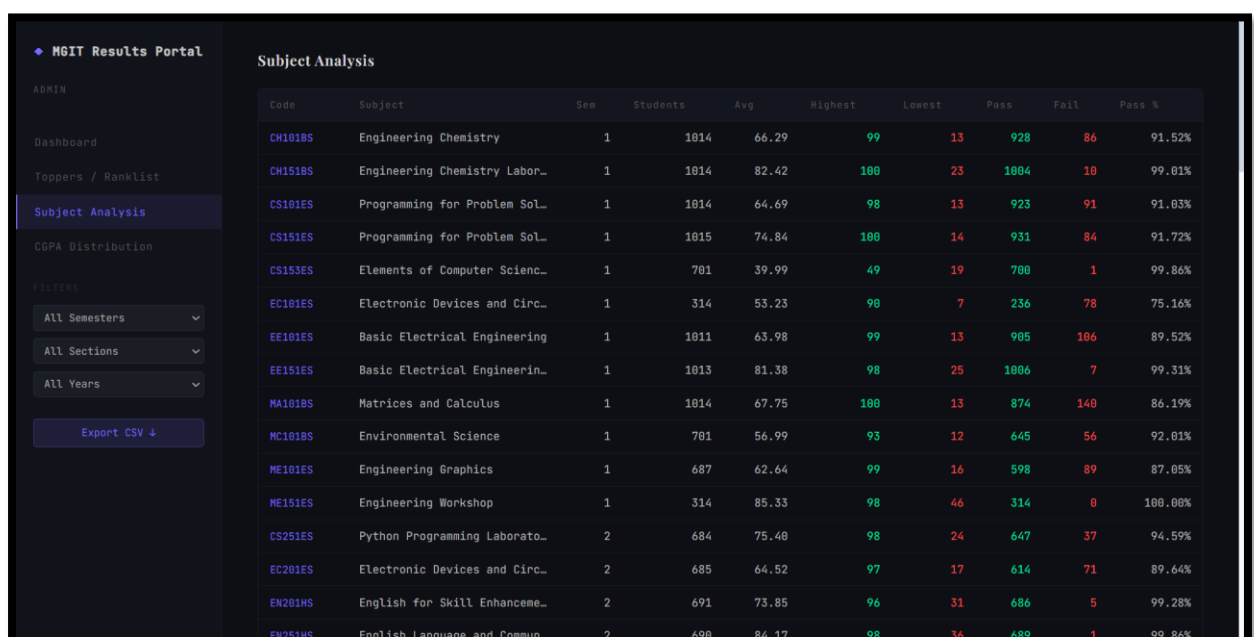


Rank	HT No	Section	Year	CGPA	Failed Subjects
#1	24261A0569	CSE-2	1	9.87	0
#2	25261A0551	CSE-1	1	9.85	0
#3	25261A05A2	CSE-2	1	9.70	0
#4	23261A0597	CSE-2	1	9.69	0
#5	24261A0580	CSE-2	1	9.65	0
#6	24261A0592	CSE-2	1	9.63	0
#6	24261A05U4	CSE-5	1	9.63	0
#8	24261A05L9	CSE-4	1	9.62	0
#9	24261A0595	CSE-2	1	9.60	0
#10	24261A05P9	CSE-4	1	9.57	0
#11	25261A0579	CSE-2	1	9.55	0
#11	25261A0555	CSE-1	1	9.55	0
#11	25261A0587	CSE-2	1	9.55	0
#14	23261A0553	CSE-1	1	9.53	0
#14	24261A05J8	CSE-3	1	9.53	0
#14	24261A05H4	CSE-3	1	9.52	0

Figure 6.9: Admin Toppers — CGPA-ranked student list with section, year, and semester filters

6.10 Admin Subject Analysis

The Subject Analysis tab shows a row per subject with average marks, highest and lowest scores, pass count, fail count, and pass percentage. A semester filter reduces the view to a specific term.



Code	Subject	Sem	Students	Avg	Highest	Lowest	Pass	Fail	Pass %
CH101BS	Engineering Chemistry	1	1014	66.29	99	13	928	86	91.52%
CH151BS	Engineering Chemistry Labor...	1	1014	82.42	100	23	1004	10	99.01%
CS101ES	Programming for Problem Sol...	1	1014	64.69	98	13	923	91	91.03%
CS151ES	Programming for Problem Sol...	1	1015	74.84	100	14	931	84	91.72%
CS153ES	Elements of Computer Scienc...	1	701	39.99	49	19	700	1	99.86%
EC101ES	Electronic Devices and Circ...	1	314	53.23	90	7	236	78	75.16%
EE101ES	Basic Electrical Engineering	1	1011	63.98	99	13	905	106	89.52%
EE151ES	Basic Electrical Engineerin...	1	1013	81.38	98	25	1006	7	99.31%
MA101BS	Matrices and Calculus	1	1014	67.75	100	13	874	140	86.19%
MC101BS	Environmental Science	1	701	56.99	93	12	645	56	92.01%
ME101ES	Engineering Graphics	1	687	62.64	99	16	598	89	87.05%
ME151ES	Engineering Workshop	1	314	85.33	98	46	314	0	100.00%
CS251ES	Python Programming Laborato...	2	684	75.40	98	24	647	37	94.59%
EC201ES	Electronic Devices and Circ...	2	685	64.52	97	17	614	71	89.64%
EN201HS	English for Skill Enhanceme...	2	691	73.85	96	31	686	5	99.28%
EN251HS	English Language and Commun...	2	498	84.17	98	34	480	1	99.84%

Figure 6.10: Admin Subject Analysis — Per-subject statistics table with pass percentage

6.11 Admin CGPA Distribution

The CGPA Distribution tab displays a horizontal bar chart with 0.5-step CGPA buckets. Summary cards below aggregate students into four broad performance bands with percentage share of the total cohort.

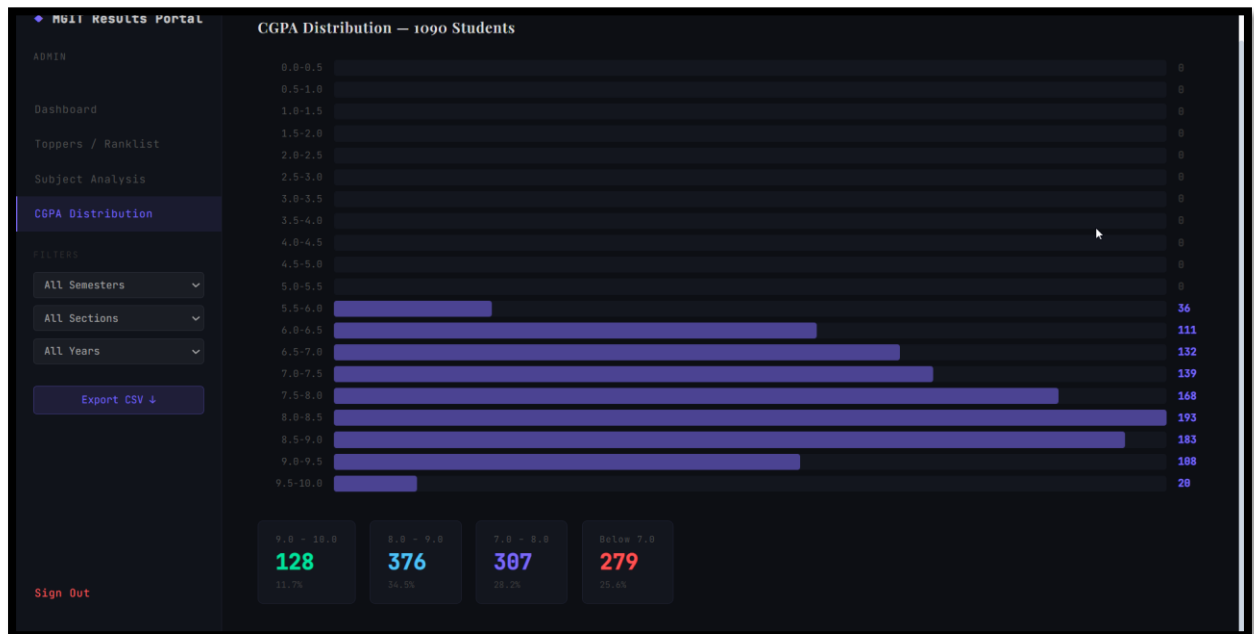


Figure 6.11: Admin CGPA Distribution — Histogram with 0.5-step buckets and summary band cards

7. ENHANCEMENTS

7.1 Student Name Integration

The current dataset contains no student names — students are identified solely by their hall ticket number. The database schema includes a name column in the students table that is currently unpopulated. Once the department provides a name mapping, the import script can be extended to populate this field and all student-facing screens can display a personalised greeting alongside the HT number.

7.2 Password Change Feature

Students currently use a default password that cannot be changed through the application. Adding a password change route and a form on the dashboard would allow students to set a personal password, improving both usability and security.

7.3 Projection Model Improvement

The current SGPA projection uses simple linear regression, which assumes a linear trend. A polynomial regression or weighted regression model (where more recent semesters carry greater weight) would produce more accurate projections for students whose performance has been non-linear. This would be a natural extension once more semester data is available.

7.4 Email Notifications

Automated email notifications could alert students when marks are updated by the admin or when a new semester's results are imported. This would eliminate the need for students to check the portal manually and would increase platform engagement.

7.5 Mobile Application

A React Native or Flutter mobile application would provide a more native experience for the student role. Push notifications for result updates and direct camera access for document scanning would become possible through a mobile client.

7.6 Attendance Integration

Integrating attendance data alongside result data would allow the admin dashboard to correlate attendance percentages with academic performance, providing richer insights into the factors that affect student outcomes across sections and subjects.

8. CONCLUSION

The Result Analysis System successfully demonstrates the practical application of Node.js, Express.js, MySQL, and React in building a production-grade, multi-role web application. Developed as a two-person team project, it is fully deployed and publicly accessible, making it a functional deliverable beyond a purely academic exercise.

The system manages result data for 1,090 students across 35,582 result entries, supports two distinct user roles with JWT-based role-specific access control, automates SGPA and CGPA calculation, implements a linear regression-based SGPA projection for advanced students, generates PDF marksheets, enables batch rank computation, and provides comprehensive department-wide analytics across sections and semesters.

The student dashboard's five-tab design — covering an overview with batch rank, a searchable results browser, a GPA overview with projection charts, a failed subjects tracker, and marksheet downloads — demonstrates how a complex feature set can be organised for clarity and usability. The linear regression projection in particular represents a meaningful application of statistical reasoning to a practical academic problem.

Key engineering concepts applied include RESTful API design, stateless JWT authentication, relational database schema design with foreign key constraints and unique indexes, bcrypt password security, middleware-based route protection, cloud deployment across multiple platforms, and collaborative Git-based development. The project encountered and resolved real-world challenges including MySQL strict mode compatibility, CORS configuration for cross-origin deployment, PDF generation library incompatibility with serverless hosting, and SPA routing on static CDN hosting.

The codebase is designed for extensibility. The modular route structure, shared utility functions, and clean separation between the frontend and backend make it straightforward to add the faculty role, additional analytics, or extended reporting without disrupting existing functionality.